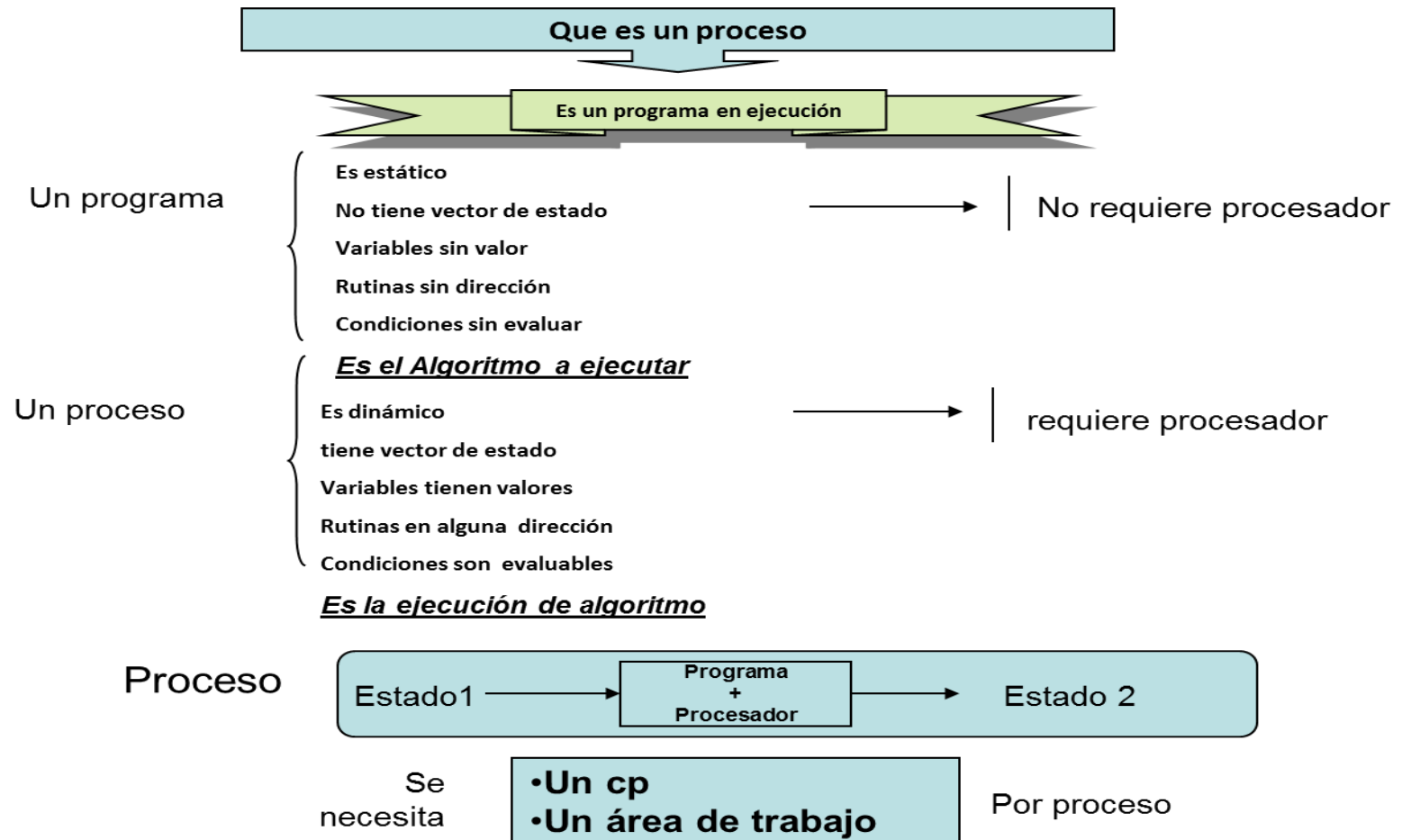




GESTION DE PROCESOS

Concepto de Proceso



GESTIÓN DE PROCESOS.

- ❖ Cada programa que se ejecuta es un proceso con recursos asignados y gestionado por el kernel.
- ❖ La gestión de procesos comprende la monitorización, detención y cambio de prioridad de los procesos
- ❖ Generalmente los procesos son gestionados automáticamente por el kernel del S.O. (son creados, ejecutados y detenidos sin la intervención del usuario).
- ❖ Algunas veces los procesos se detendrán por razones desconocidas y será necesario reiniciar el proceso.
- ❖ Otras veces algún proceso se ejecutará descontroladamente malgastando los recursos del sistema, entonces será necesaria una intervención manual del administrador para detener el proceso.

Descriptor de Proceso

Como ve el Sistema operativo a los proceso

A través se su vector de estado, en un
Descriptor de proceso o BCP

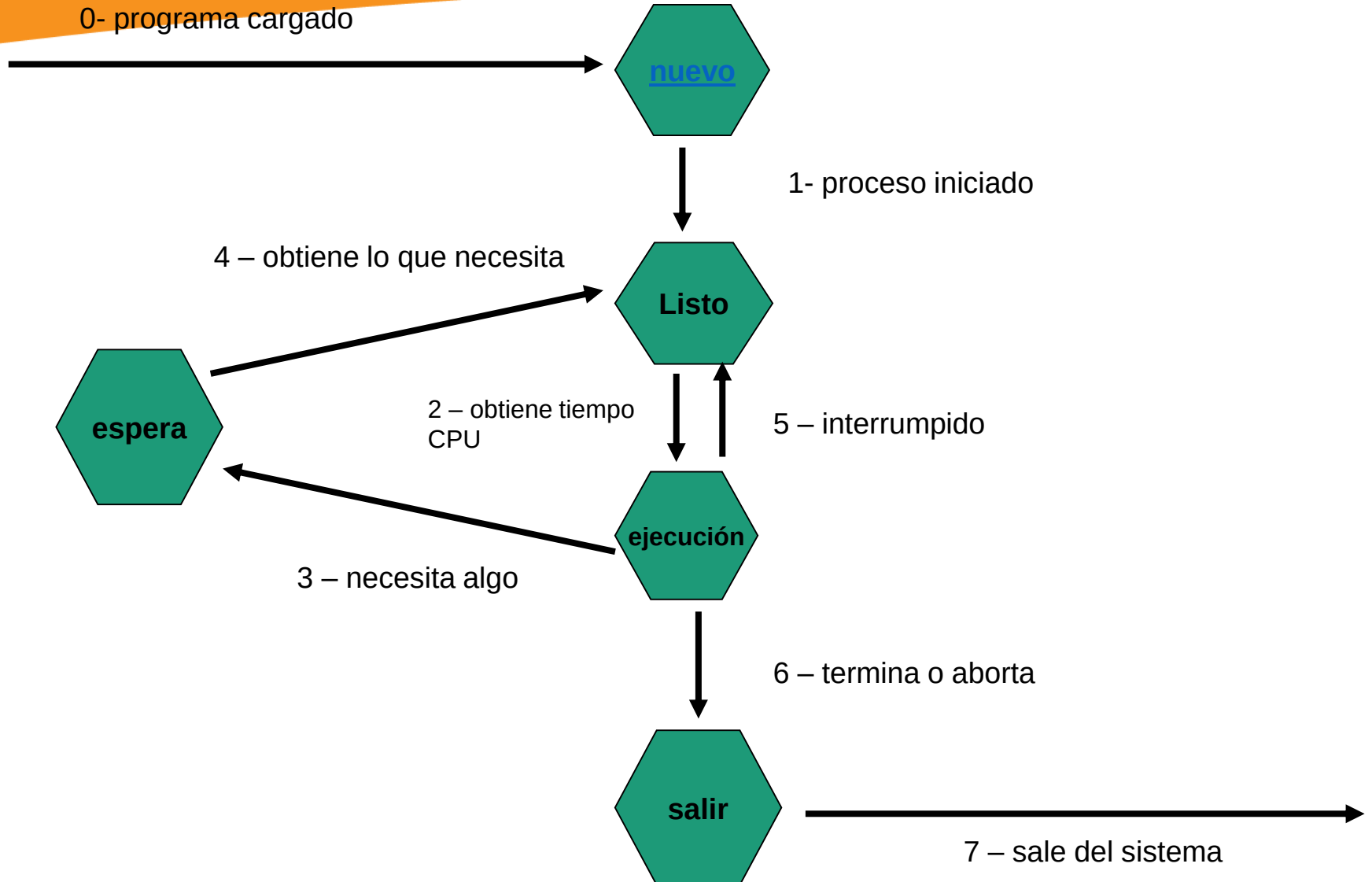
El Descriptor del
proceso
contiene información:

del Proceso
de su memoria
de estado del procesador (registros)

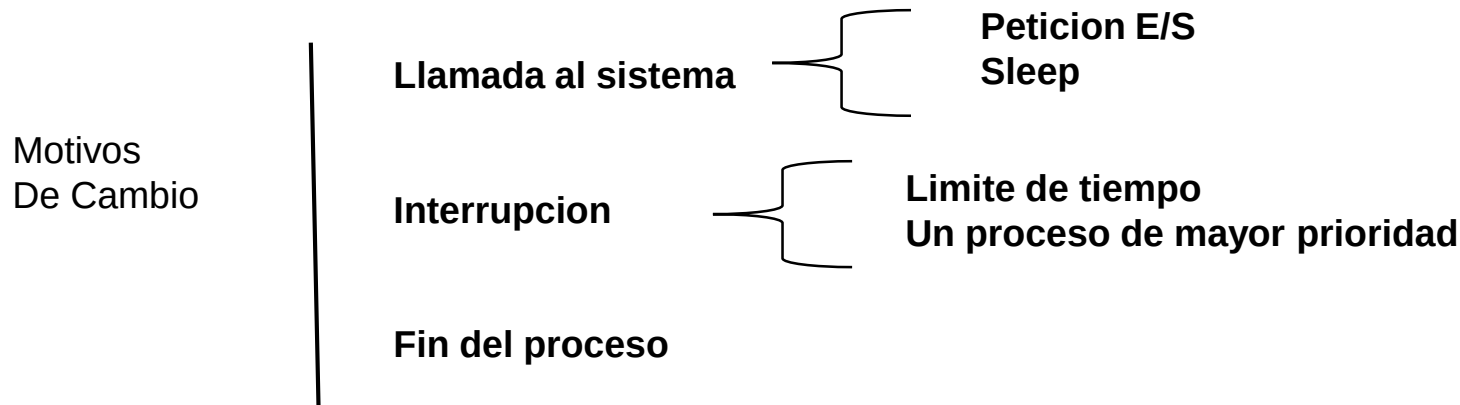
} Contexto del proceso

- Nombre del Programa
- Identificación del proceso, numero asignado por SO para identificar el proceso
- Identificador del proceso padre, o un apuntador hacia el BCP del proceso padre
- Apuntador hacia una lista de BCP hijos
- Apuntador hacia el siguiente BCP
- Información de contabilidad (contadores)
- Información de estado CPU
 - Contador de instrucciones
 - Apuntador(es) de pila
 - Registro de estado del sistema
 - Otros registros del sistema
- Descriptor de eventos, valido si el proceso esta en espera de algo
- Información de estado del proceso
- Propietario del proceso (usuario)
- Información de gestión de la memoria
- Apuntador hacia una cola
- Apuntador hacia una cola de eventos

Estados y Transiciones de los procesos



CAMBIO DE PROCESO



1. Selecciona el Siguiente proceso
↓
Scheduler
2. Cambiar el contexto (de ejecucion)
↓
Dispatcher

El cambio de proceso consume tiempo

MINIMIZAR LOS CAMBIOS DE PROCESOS

Procesos

Existe una función "**scheduler**" o "**planificador**" que en función de la política , se encarga de **seleccionar a un proceso** de la lista de "listo para ejecutarse".

Un vez seleccionado el proceso a ejecutarse por el "planificador", se llama a la función "**dispatcher**" que se encarga de **pasar a estado de ejecución** a dicho proceso, restaurando su información de contexto.

JERARQUÍA DE LOS PROCESOS

La jerarquía de los procesos, es un concepto aplicado únicamente a SO. UNIX:

- Un proceso puede crear otro proceso, por medio de una llamada al sistema (fork).
- El proceso que se crea, se denomina proceso HIJO, entre tanto el proceso que lo creo, se denomina PADRE.
- Un proceso PADRE, puede tener múltiples procesos HIJO, sin embargo éste último sólo tendrá un proceso PADRE.
- Se crea entonces, un ÚNICO “árbol” de procesos en el SO.
- Windows no tiene jerarquía de procesos pero cuando un proceso crea a otro recibe una referencia (identificador) que le sirve para controlar a su hijo..

API de Unix para manejo de Procesos

- ☐ Unix proporciona un número de llamadas al sistema para creación y control de procesos, que incluye:
 - ☐ Fork
 - ☐ exec:
 - ☐ vfork:
 - ☐ clone():
 - ☐ wait:.
 - ☐ waitpid:
 - ☐ sleep:
 - ☐ kill:
 - ☐ abort:
 - ☐ getpid:
 - ☐ getppid:
 - ☐ getuid:
 - ☐ getpgrp:

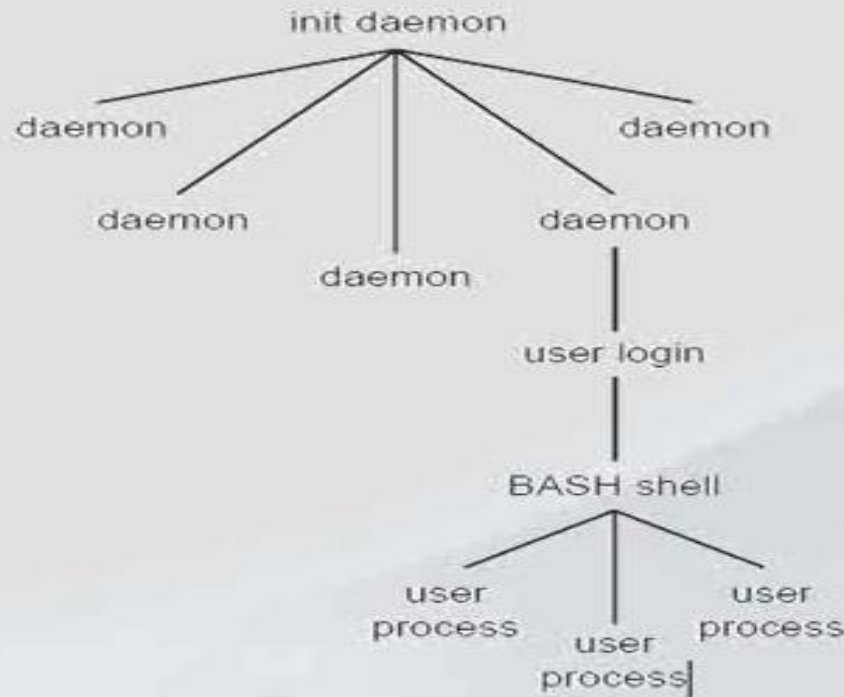
ACTIVIDAD INVESTIGUE Y RELLENE LAS DEFINICIONES TIEMPO 10 MINUTOS

Procesos

- UNIX/Linux es un sistema multiproceso que permite la ejecución de varios procesos de forma simultánea
- Requiere de otro programa llamado “Planificador o scheduler” que permite gestionar que proceso entra a ejecutarse en cada instante en la CPU, ya que esta es un recurso limitado.
- Todo proceso nace cuando algún otro proceso ejecuta la llamada al sistema clone. Una de las muchas formas de llamar a clone es a través de la función fork.
- clone crea un duplicado idéntico del proceso que la ha llamado.

JERARQUÍA DE LOS PROCESOS

Existe un proceso especial cuyo PID es 0, este se crea al arrancar el sistema y después de hacer una llamada fork se convierte en el proceso intercambiador (swapper), el proceso hijo se llama init y su PID vale 1. Este se encarga de arrancar los demás procesos que requiere el sistema.

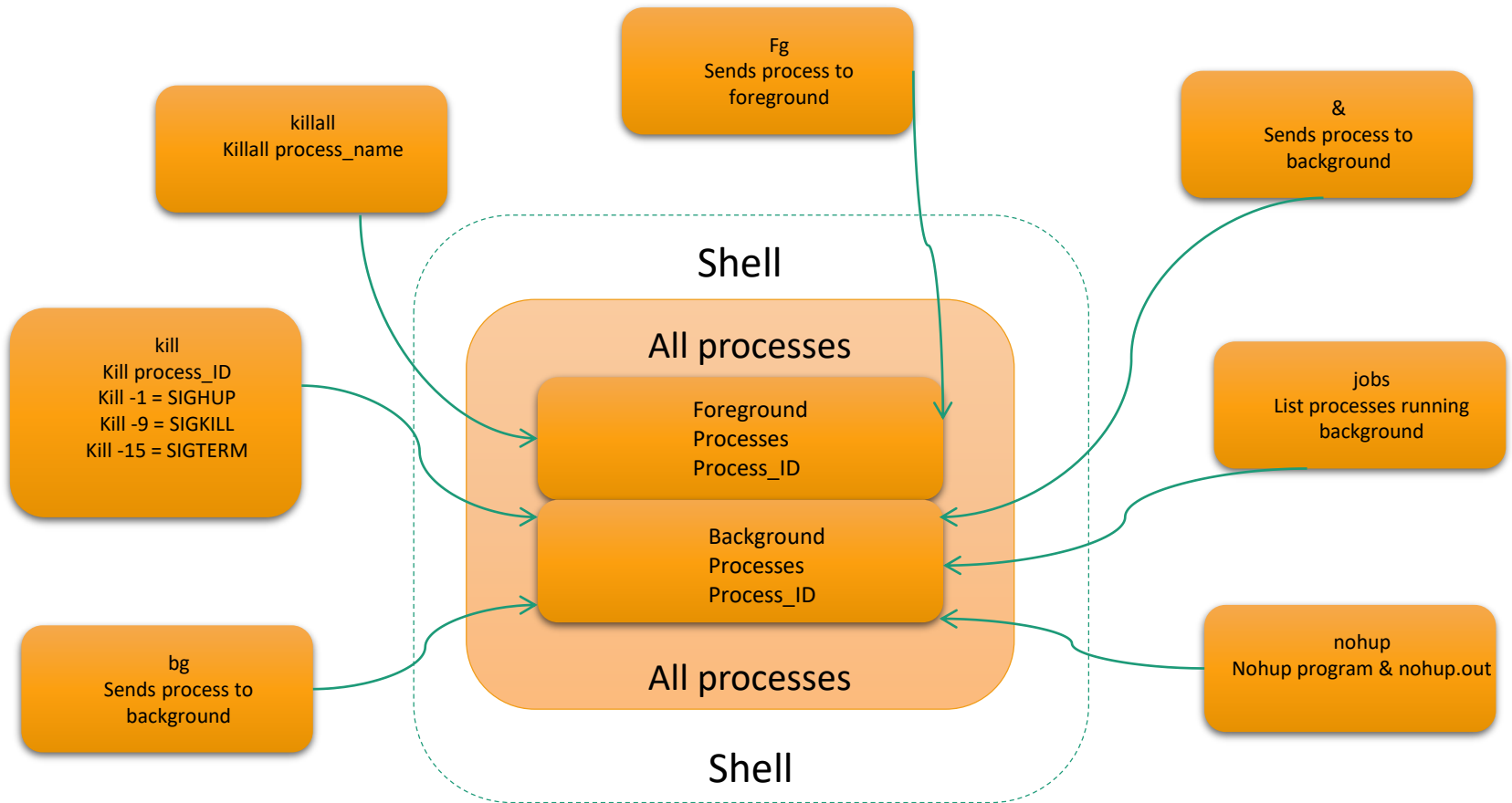


PARÁMETROS DE UN PROCESO

- **PROCESS ID (PID):** Cada proceso tiene un número asociado que se le asigna cuando es creado. Los PIDs son números enteros únicos para todos los procesos sistema.
- **USER ID & GROUP ID:** Cada proceso tiene que tener asociado unos privilegios que limiten el acceso al sistema de ficheros. Estos privilegios quedan determinados por el user ID y group ID del usuario que creo el proceso.
- **PARENT PROCESS:** Todo proceso es creado por otro proceso, el proceso padre (parent process). El primer proceso iniciado por el kernel cuando el sistema arranca es el programa init. Este proceso tiene el PID 1 y es el padre de todos los procesos del sistema.

PARÁMETROS DE UN PROCESO

- **PARENT PROCESS ID:** El PID del proceso que inicio el proceso hijo.
- **ENVIROMENT:** Cada proceso mantiene una lista de variables y sus correspondientes valores. El conjunto de estas variables recibe el nombre de process enviroment. Normalmente el entorno de un proceso hijo se hereda del proceso padre a menos de que se indique de otra forma.
- **CURRENT WORKING DIRECTORY:** Cada proceso tiene asociado un directorio por defecto, donde el proceso leerá/escribirá archivos, a menos que se le especifique explícitamente lo contrario.
- **NICE NUMBER:** Permite al usuario modificar la prioridad de ejecución de un proceso.



EJECUCIÓN EN 1º Y 2º PLANO

- ❖ Un proceso ejecutado en 1º plano, se ejecuta bloqueando el terminal desde el que se lanzó.
- ❖ Un proceso se lanza en 1º plano simplemente introduciendo su nombre en el terminal y pulsando intro.

\$Firefox

- ❖ Un proceso ejecutado en 2º plano, se ejecuta sin bloquear el terminal desde el que se lanzó.
- ❖ Los programas se pueden iniciarse en 2º plano añadiendo el carácter & al final del comando.

\$firefox &

[1] 1748

El sistema operativo le asigna el trabajo 1 (indicado mediante [1]) y el PID 1748.

EJECUCIÓN EN 1º Y 2º PLANO

- ❖ Cuando un proceso se inicia en segundo plano, se crea un trabajo (job), al cual se le asigna un número entero, empezando por 1 y numerando secuencialmente.
- ❖ Cuando un proceso se ejecuta en segundo plano (background), la única manera de comunicarse con el proceso es mediante el envío de mensajes (signals).
- ❖ Se puede mover un programa ejecutado en primer plano al segundo plano, deteniéndolo escribiendo Ctrl-Z y después reiniciando el proceso en 2º plano, mediante la orden bg.

OPERACIONES CON PROCESOS EN PRIMER PLANO

Con un proceso en 1º plano podemos realizar dos acciones desde el terminal que tiene asociado:

❖ **Matar el proceso:** C-c (Ctrl-c) se cancela el proceso y se liberan todos los recursos que tuviera asignados. Esta acción realmente envía una señal de interrupción al proceso, indicándole que tiene que detenerse.

Sólo podremos matar procesos sobre los que tengamos permiso. Si intentamos matar el proceso de otro usuario el sistema no nos lo permitirá, salvo a root.

❖ **Parar el proceso:** C-z (Ctrl-z) . En este caso sólo se detiene la ejecución del proceso, conservando su estado y sus recursos para poder continuar en el momento que se dé la orden adecuada.

CONTROL DE TRABAJOS DESDE EL TERMINAL: BG, FG, JOBS

- ❖ Cuando lanzamos un proceso en segundo plano obtenemos un PID y un número de trabajo. El PID es el número de proceso, que es utilizado por el S.O. para identificar de forma única al proceso. En cambio el número de trabajo es un identificador de uno o varios procesos correspondientes a un usuario.
- ❖ Anteriormente vimos que C-z detiene (suspende) un proceso y lo deja en segundo plano. También vimos como podíamos enviarle un señal para que continuara su ejecución en segundo plano. Ahora lo que vamos ver son los mecanismos para realizar una gestión más completa de esos trabajos.

CONTROL DE TRABAJOS DESDE EL TERMINAL: BG, FG, JOBS

❖ bg (background)

Sintaxis: bg [jobspec]

Mueve el trabajo jobspec a segundo plano, como si se hubiese iniciado con &. Si el trabajo especificado está detenido, el comando bg lo reiniciará en segundo plano.

Ejemplo:

CONTROL DE TRABAJOS DESDE EL TERMINAL: BG, FG, JOBS

❖ fg (foreground)

Sintaxis: fg [jobspec]

Se utiliza para traer a primer plano un trabajo que está en segundo plano, bien esté activo o bien esté detenido.

Ejemplo:

\$ sleep 200 (y pulsar C-z)	Ejecutamos en 1° plano y detenemos
\$ sleep 300 (y pulsar C-z)	Ejecutamos en 1° plano y detenemos
\$ fg 1 (y pulsar C-z)	Pasamos a 1° plano y detenemos
\$ bg 2	Pasamos a 2° plano
\$ fg 1	Pasamos a 1° plano

CONTROL DE TRABAJOS DESDE EL TERMINAL: BG, FG, JOBS, SHOFT

❖ jobs

Sintaxis: jobs [options] [jobspec]

Lista todas las tareas activas. Si se incluye jobspec únicamente listará la información sobre esas tareas.

Con la orden jobs podemos obtener una lista de los trabajos que hemos lanzado en el sistema. La orden jobs se utiliza como:

Opciones:

-l Lista los PID de las tareas.

Ejemplo:

```
elara@elara-desktop:~$ sleep 600 &
[1] 28185
elara@elara-desktop:~$ sleep 700 &
[2] 28221
elara@elara-desktop:~$ jobs
[1]-  Running                  sleep 600 &
[2]+  Running                  sleep 700 &
elara@elara-desktop:~$
```

Entre corchetes tenemos el número de trabajo, y los signos + y menos indican:

- + El trabajo es el primero de la lista
- El trabajo es el segundo de la lista

CONTROL DE TRABAJOS DESDE EL TERMINAL: BG, FG, JOBS

❖ **Nohup*******

Sintaxis: nohup orden [argumentos]

La orden nohup lanza un proceso y lo independiza del terminal que estamos usando. Los procesos se organizan de forma jerárquica, de forma que si abandonamos la shell que nos conectó al sistema (abandonamos la sesión de trabajo) automáticamente se matarán todos los procesos que dependan de ella. Pero en muchas ocasiones no puede interesar lanzar un proceso y dejarlo en ejecución aun cuando hayamos cerrado la sesión de trabajo. Para esto se usa la orden nohup.

USOS DEL NOHUP*****

nohup df -k

1. Ejecute nohup con un comando de ping en un segundo plano

1. Elimine el proceso

2. Redirija el resultado de un comando a un archivo

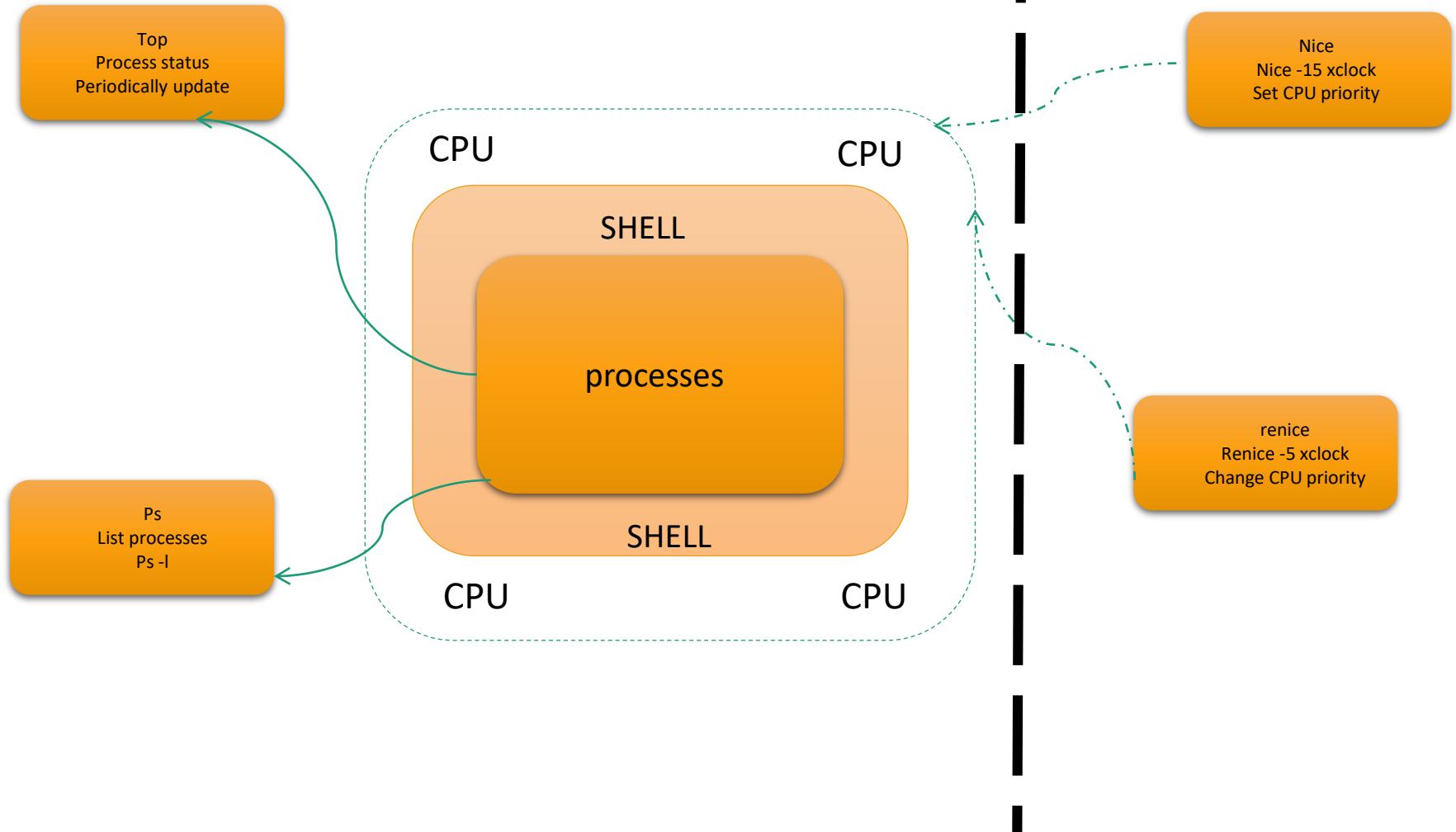
ACTIVIDAD INDIVIDUAL 10 Minutos

CAMBIAR LA PRIORIDAD DE LOS PROCESOS: NICE & RENICE

- ❖ Cada proceso tiene una determinada prioridad de ejecución, al necesitar +/- tiempo de CPU que otros.
- ❖ Normalmente la prioridad de los procesos es gestionada automáticamente por el kernel.
- ❖ No obstante, LINUX ofrece la posibilidad de modificar estas prioridades y favorecer la ejecución de ciertos procesos respecto a otros.
- ❖ La prioridad de un proceso puede determinarse examinando la columna PRI en los resultados de los comandos `top` y `ps -l`. Los valores mostrados son relativos, cuanto mayor es el PRI, mayor es el tiempo de CPU dedicado por el kernel a ese proceso.

Manage Processes

Manage CPU power



Ordenes relativas a procesos: ps

- Campos de salida:
 - UID
 - %CPU
 - START
 - TTY
 - TIME
 - PRI
 - NI(Valor del NICE)
 - SIZE
 - RSS
 - STAT (Estado del proceso):
 - D:
 - R:
 - S:
 - Z:
 - T:
 - W:
 - N:

ACTIVIDAD INVESTIGUE Y RELLENE LAS DEFINICIONES TIEMPO 20 MINUTOS

CAMBIAR LA PRIORIDAD DE LOS PROCESOS: NICE & RENICE

```

elara@elara-desktop: ~
Archivo Editar Ver Terminal Solapas Ayuda
top - 08:05:47 up 10 min, 2 users, load average: 0.30, 0.64, 0.56
Tasks: 116 total, 1 running, 115 sleeping, 0 stopped, 0 zombie
Cpu(s): 4.4%us, 2.0%sy, 0.0%ni, 93.3%id, 0.0%wa, 0.3%hi, 0.0%si, 0.0%st
Mem: 515580k total, 427036k used, 88544k free, 40712k buffers
Swap: 192740k total, 0k used, 192740k free, 232332k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 5137 root        20   0 38124 12m 6368 S   4.6   2.5   0:13.30 Xorg
 5768 elara      20   0 74984 20m 11m S   1.3   4.0   0:01.62 gnome-terminal
 5653 elara      20   0 25736 11m 8208 S   0.7   2.3   0:02.08 nm-applet
 5788 elara      20   0 2308 1116 852 R   0.7   0.2   0:00.20 top
 5582 elara      20   0 28620 5808 3372 S   0.3   1.1   0:00.95 pulseaudio
   1 root        20   0 2844 1692 544 S   0.0   0.3   0:03.06 init
   2 root        15  -5     0     0     0 S   0.0   0.0   0:00.00 kthreadd
   3 root         RT  -5     0     0     0 S   0.0   0.0   0:00.00 migration/0
   4 root        15  -5     0     0     0 S   0.0   0.0   0:00.00 ksoftirqd/0
   5 root         RT  -5     0     0     0 S   0.0   0.0   0:00.00 watchdog/0
   6 root        15  -5     0     0     0 S   0.0   0.0   0:00.08 events/0
   7 root        15  -5     0     0     0 S   0.0   0.0   0:00.04 khelper
  41 root        15  -5     0     0     0 S   0.0   0.0   0:00.60 kblockd/0
  44 root        15  -5     0     0     0 S   0.0   0.0   0:00.00 kacpid
  45 root        15  -5     0     0     0 S   0.0   0.0   0:00.00 kacpi_notify
 110 root        15  -5     0     0     0 S   0.0   0.0   0:00.02 kseriod
 148 root        20   0     0     0     0 S   0.0   0.0   0:00.00 pdflush
    
```

top

ps -l

```

elara@elara-desktop: ~
Archivo Editar Ver Terminal Solapas Ayuda
elara@elara-desktop:~$ ps -l
 F S   UID    PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
 0 S   1000    5771 5768   0  80   0 - 1402 -  pts/0      00:00:00 bash
 0 R   1000    5792 5771   0  80   0 - 607 -  pts/0      00:00:00 ps
elara@elara-desktop:~$ 
    
```

CAMBIAR LA PRIORIDAD DE LOS PROCESOS: NICE & RENICE

- ❖ El parámetro que permite al usuario modificar la prioridad de ejecución de un proceso recibe el nombre de nice number.
- ❖ Por defecto los procesos poseen un valor de nice igual a cero. Con este valor el kernel no modifica la prioridad del proceso.
- ❖ El parámetro nice puede tomar valores comprendidos entre **-20 y +19**. Cualquier usuario puede aumentar el valor de nice, y bajar la prioridad del proceso, pero únicamente el usuario root puede asignar números negativos a nice, e incrementar la prioridad y por lo tanto el tiempo de CPU.



CAMBIAR LA PRIORIDAD DE LOS PROCESOS: NICE & RENICE

`nice -n -5 yes`

`ps -el`

`renice 7 numero proceso`

`Kill -9 numero proceso`

CAMBIAR LA PRIORIDAD DE LOS PROCESOS: NICE & RENICE

❖ renice

Sintaxis: renice [+|- nicenumber] [options] targets

Este comando permite modificar el parámetro nice de un proceso ya iniciado.

Opciones:

- u Interpreta targets como un nombre de usuario. Cambia el parámetro nice a todos los procesos propietarios del usuario especificado.
- p Interpreta targets como un PID (comportamiento por defecto).

Ejemplos:

renice 19 501 Baja la prioridad del proceso con PID 501 incrementando su número nice al máximo.

renice -20 -u pepe 501 Aumenta la prioridad de todos los procesos del usuario pepe y el proceso con PID 501. Únicamente el root puede ejecutar el comando renice con valores negativos.

ENVIAR MENSAJES A UN PROCESO: KILL

- ❖ Cada proceso que se ejecuta en el sistema está alerta de los mensajes enviados por el kernel o por el usuario.
- ❖ Las órdenes kill y killall se utilizan para enviar señales a un proceso.
- ❖ Estos mensajes (signals) son números enteros predefinidos y conocidos por los procesos.
- ❖ Cuando un proceso recibe un mensaje, este realiza una determinada acción.
- ❖ Existen 30 diferentes señales definidas en LINUX. Cada señal tiene un nombre y un entero.

ORDEN KILL: ENVÍO DE SEÑALES A PROCESOS

Kill es un comando unix/Linux que permite enviar señales a uno o varios procesos del sistema a través de la Shell (intérprete de órdenes o intérprete de comandos).

La principal función para utilizar este comando es cuando un servicio o cuando un proceso se sale de control o cuando entra en un ciclo infinito causando que el servidor o el sistema se ponga lento o que deja de responder al ocupar demasiados recursos del sistema como la memoria o el espacio en disco. En estos casos es necesario terminar o matar dicho proceso.

Para utilizar el comando en Linux y matar algún proceso solo debes indicar en identificador de proceso, este puede ser llamado PID o Process ID, para conocer el PID de un programa o proceso puedes utilizar el comando **ps** para obtener una lista de los procesos que están en ejecución.

- El listado completo de señales disponibles puede visualizarse ejecutando kill-l:

ENVIAR MENSAJES A UN PROCESO: KILL

Nombre	Entero	Descripción
SIGHUP	1	Colgar. Esta señal es enviada automáticamente cuando un modem se desconecta. También se usa por muchos demonios para forzar la relectura del archivo de configuración. Por ejemplo, en los procesos init y inetd.
SIGINT	2	Parar el proceso y desaparece. Esta señal se envía con la secuencia de teclas Ctrl-C.
SIGKILL	9	Kill. Mata un proceso incondicionalmente e inmediatamente. Enviar esta señal es un método drástico de terminar el proceso, ya que no se puede ignorar
SIGTERM	15	Terminar. Mata un proceso de forma controlada.
SIGCONT	18	Continuar. Cuando un proceso detenido recibe esta señal continúa su ejecución.
SIGSTOP	19	Parar, pero preparado para continuar. Esta señal se envía con la secuencia de teclas Ctrl-Z.

- En este ejemplo el PID es mostrado como un número al inicio de cada línea. Como vemos el proceso “bash” tiene el PID 2541.

```
$ ps
PID TTY TIME CMD
2541 pts/0 00:00:00 bash
2590 pts/0 00:00:00 ps
```

- Si quisieras detener este proceso lo que tienes que hacer es ejecutar **el comando kill** y el número de PID 2541 de la siguiente forma:

```
$ kill 2541
```

- Hay ocasiones que un proceso puede no responder a esta señal. Si se desea matar ese proceso se puede enviar la señal 9) SIGKILL agregando un -9 como parámetro al comando kill, así:

```
$ kill -9 2541
```

Otro ejemplo práctico del uso del comando **kill** es cuando requieres terminar o matar varios procesos a la vez, para conocer los PIDs de algún programa podemos usar el comando **ps** y enviar la salida por un pipe (tubería) hacia el **comando grep** para que nos muestre solo las líneas de texto que incluyan la palabra «apache» para luego seleccionar solo los PIDs que nos interesa detener. Así se vería la salida de estos comandos:

```
$ ps aux | grep apache
root 1489 0.0 0.3 233536 10092 ? Ss 22:11 0:00 /usr/sbin/apache2 -k start
www-data 1740 0.0 0.1 233560 6052 ? S 22:11 0:00 /usr/sbin/apache2 -k start
www-data 1741 0.0 0.1 233560 6052 ? S 22:11 0:00 /usr/sbin/apache2 -k start
www-data 1742 0.0 0.1 233560 6052 ? S 22:11 0:00 /usr/sbin/apache2 -k start
www-data 1743 0.0 0.1 233560 6052 ? S 22:11 0:00 /usr/sbin/apache2 -k start
www-data 1744 0.0 0.1 233560 6052 ? S 22:11 0:00 /usr/sbin/apache2 -k start
```

```
$ sudo kill 1740 1741 1742 1743 1744
[sudo] password for user:
```

```
$ ps aux | grep apache
root 1489 0.0 0.3 233536 10104 ? Ss 22:11 0:00 /usr/sbin/apache2 -k start
```

Trabajo en clase(TIPS)

ps aux en Linux:

Mostrar detalles específicos de un proceso con su PID (ID del proceso):

- USER:
- PID:
- %CPU:
- %MEM:.
- VSZ:
- RSS:
- TTY:
- STAT:
- START:
- TIME:
- COMMAND:

(TIPS)

```
ps axf
```

```
ps aux --sort=-%cpu,%mem
```

```
ps -eo comm
```

```
ps axf
```

```
ps aux --sort=-%cpu,%mem
```

```
ps -o ppid,pid,uname,comm -ppid=1
```

```
ps -ef | wc -l
```

(TIPS)

- Establezca la diferencia entre Proceso, Servicio, Daemon
- Establezca la Diferencia entre Script y Programa
- Establezca la Diferencia entre JOBS y Proceso
- Cuales son los tipos de daemons en linux

(TIPS)

- Que se entiende por proceso ocioso
- Proceso Huerfano y Zombie
- Criterios para los algoritmos de Planificación(6)
- Gestion de procesos en ANDROID
 - Parejas enviar por correo electronico